

Manual Técnico

RD Clima — Gestão de Instalações AC

Unidade Curricular de Portfólio — Licenciatura em Engenharia Informática

Escola Superior de Tecnologia de Setúbal — Instituto Politécnico de Setúbal

Estudante: Filipe Dordio (n.º 202200035)

Coordenador: Prof. Rui César das Neves

16 de outubro de 2026

Índice

1. Introdução 3
2. Arquitetura 3
3. Tecnologias 4
4. Estrutura do código 4
5. Modelo de dados 5
6. Ciclo de vida de um trabalho 6
7. Segurança 6
8. Lógica na base de dados 7
9. Notificações push e PWA 7
10. Instalação, configuração e deploy 8
    10.1 Requisitos 8
    10.2 Base de dados 8
    10.3 Notificações 8
    10.4 Aplicação 8
    10.5 Publicação na Vercel 9
11. Testes automáticos 9
12. Limitações conhecidas e trabalho futuro 9

1. Introdução

A RD Clima é uma aplicação web progressiva (PWA) que a empresa RD Clima usa para agendar e registar instalações e manutenções de ar condicionado, bombas de calor, solar térmico e hidráulica. O administrador marca os trabalhos no calendário e atribui-os aos técnicos. No telemóvel, os técnicos veem os trabalhos do dia e registam o resultado com foto e notas.

A aplicação é uma PWA em React e TypeScript, com o Supabase como backend e alojamento na Vercel. Grande parte das regras de negócio fica na base de dados: as permissões por perfil são garantidas por Row Level Security e o histórico e os aparelhos são mantidos por triggers. Este manual descreve a arquitetura, o modelo de dados, a segurança e os passos para instalar, configurar, publicar e testar o projeto.

Existem dois perfis de utilizador: o **administrador** (o dono da empresa), que gere trabalhos, clientes, técnicos e manutenções, e o **técnico**, que vê e regista os trabalhos que lhe foram atribuídos.

2. Arquitetura

A aplicação não tem um servidor próprio. O browser descarrega a PWA da Vercel e comunica diretamente com o Supabase através do `supabase-js`. A única lógica no servidor é a Edge Function `send-push`, porque o envio de notificações precisa de chaves que não podem estar no browser.

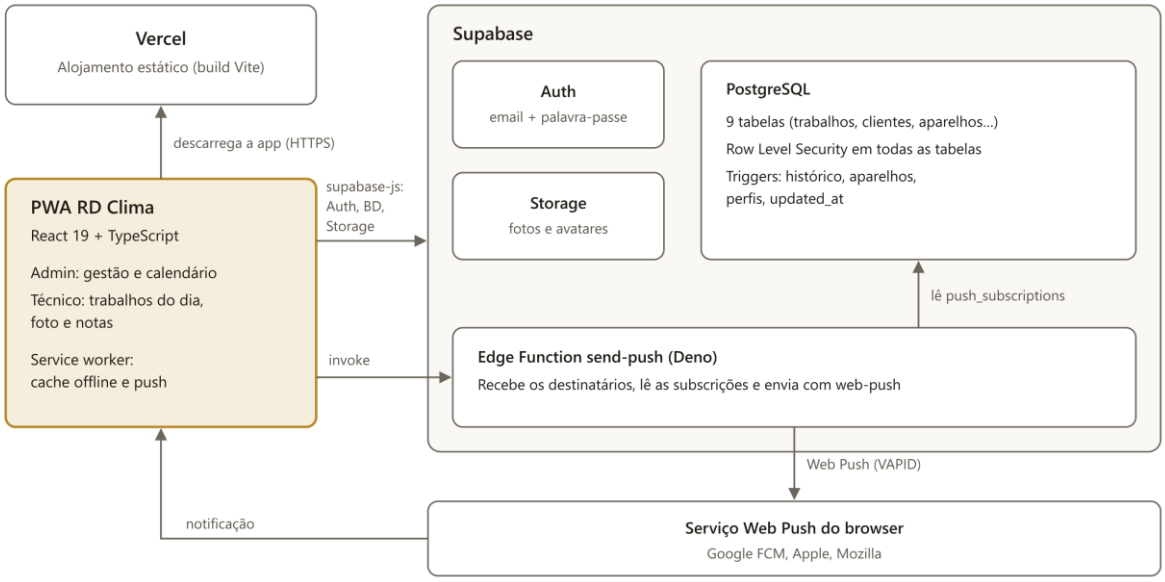


Figura 1 — Componentes da solução e comunicação entre eles

Componente	Responsabilidade
PWA (React)	Interface, navegação por perfil, formulários, calendário; instalável no telemóvel
Service worker (`src/sw.ts`)	Pré-cache dos ficheiros da app, receção de notificações push, abertura da app ao tocar na notificação, aviso de nova versão
Supabase Auth	Contas com email e palavra-passe; sessão JWT usada em todos os pedidos
PostgreSQL	Dados, regras de acesso (RLS) e regras automáticas (triggers)

Componente	Responsabilidade
Supabase Storage	Fotos das instalações e avatares
Edge Function `send-push`	Envia notificações Web Push às subscrições dos utilizadores indicados
Vercel	Alojamento estático; `vercel.json` reencaminha todas as rotas para `index.html` (SPA)

3. Tecnologias

Camada	Tecnologias
Frontend	React 19, TypeScript, Vite, React Router, Tailwind CSS 4, date-fns
Backend e base de dados	Supabase: PostgreSQL (RLS e triggers), Auth, Storage, Edge Functions
PWA e notificações	vite-plugin-pwa (service worker Workbox), Web Push (VAPID)
Testes e qualidade	Vitest, Testing Library, jsdom, ESLint, typescript-eslint
Alojamento e versões	Vercel, Git e GitHub

Algumas escolhas e os seus motivos:

- **Supabase em vez de uma API própria.** Dá autenticação, base de dados relacional e ficheiros sem manter um servidor. A segurança fica no PostgreSQL (RLS), onde não pode ser contornada a partir do browser.
- **PWA em vez de app nativa.** Uma única base de código serve o computador do administrador e os telemóveis dos técnicos, instala-se a partir do browser e atualiza-se sem lojas de aplicações.
- **Avisos ao cliente por links `wa.me` e `mailto`.** Abrem o WhatsApp ou o email do técnico com a mensagem já escrita, sem APIs pagas.

4. Estrutura do código

O código do frontend está em `src/` e tudo o que diz respeito à base de dados está em `supabase/`. São cerca de 6 800 linhas de TypeScript e SQL.

Caminho	Conteúdo
`src/App.tsx`	Rotas (React Router) e proteção de cada rota por perfil com `ProtectedRoute`
`src/pages/`	Um ecrã por ficheiro: Login, Hoje, calendários do admin e do técnico, Clientes, Técnicos, Serviços, Manutenções, Por remarcar, Estatísticas, Configurações, Perfil
`src/components/`	Componentes reutilizáveis: `JobForm`, `JobCard`, `CalendarView`, `MarkInstalledModal`, `MarkNotInstalledModal`, `CameraCapture`, `ProximasManutencoes`, `NotifyClienteButtons`, `Layout`
`src/contexts/AuthContext.tsx`	Sessão do Supabase Auth e perfil do utilizador autenticado
`src/lib/`	Lógica sem interface: cliente Supabase, subscrição e envio de push, links de WhatsApp e email (`notify.ts`), datas de manutenção (`manutencao.ts`), atribuição de técnicos
`src/types/index.ts`	Tipos TypeScript do domínio e etiquetas de apresentação

Caminho	Conteúdo
src/sw.ts	Service worker
supabase/schema.sql	Esquema base (equivalente ao estado da v8)
supabase/migration_v2.sql … v13.sql	Migrações incrementais, aplicadas por ordem
supabase/functions/send-push/	Edge Function (Deno) de envio de notificações
*.test.ts(x)	Testes Vitest, junto ao código que testam

5. Modelo de dados

A base de dados PostgreSQL tem 9 tabelas no esquema public, todas com RLS ativo. O trabalho (jobs) é a tabela central e liga-se ao cliente, aos técnicos, ao histórico e ao aparelho.

Tabela	Guarda	Ligações principais
profiles	Nome, perfil (admin/tecnico), telemóvel, email, avatar	1:1 com auth.users
clientes	Nome, contactos, morada, localidade, dimensões, tipo de operação	—
jobs	Trabalho: tipo de serviço, modelo, multi-split, data e hora, estado, notas, foto, motivo de não instalação, próxima manutenção	cliente_id → clientes (opcional); aparelho_id → aparelhos
job_tecnicos	Atribuição N:N de técnicos a trabalhos	PK (job_id, tecnico_id)
job_historico	Eventos: criado, estado alterado, editado	job_id → jobs; autor_id → profiles
aparelhos	Equipamento instalado; última e próxima manutenção	cliente_id (opcional); job_instalacao_id único
ausencias	Férias e ausências dos técnicos	tecnico_id → profiles
push_subscriptions	Endpoint e chaves Web Push por dispositivo	user_id → profiles
modelos_equipamento	Catálogo de modelos por tipo de serviço	—

O Storage tem dois buckets: instalacoes, para as fotos dos trabalhos, e avatares, para as fotos de perfil.

Há três decisões de modelação:

- **Cliente opcional.** Um trabalho pode ser de um cliente sem ficha. O nome e a morada ficam copiados em jobs.cliente e jobs.endereco, para que o histórico não dependa da ficha.
- **Próxima manutenção com dia opcional.** Normalmente só se sabe o mês. A data é guardada no dia 1 desse mês e proxima_manutencao_dia_certo indica se o dia é exato (src/lib/manutencao.ts).
- **Vários técnicos por trabalho.** A tabela de associação job_tecnicos substituiu na v5 a antiga coluna tecnico_id.

6. Ciclo de vida de um trabalho

Um trabalho é criado como **Agendado**. O técnico fecha-o como **Instalado** ou **Não instalado**. Um trabalho não instalado aparece em "Por remarcar" até o administrador lhe dar uma nova data, o que o devolve a Agendado.

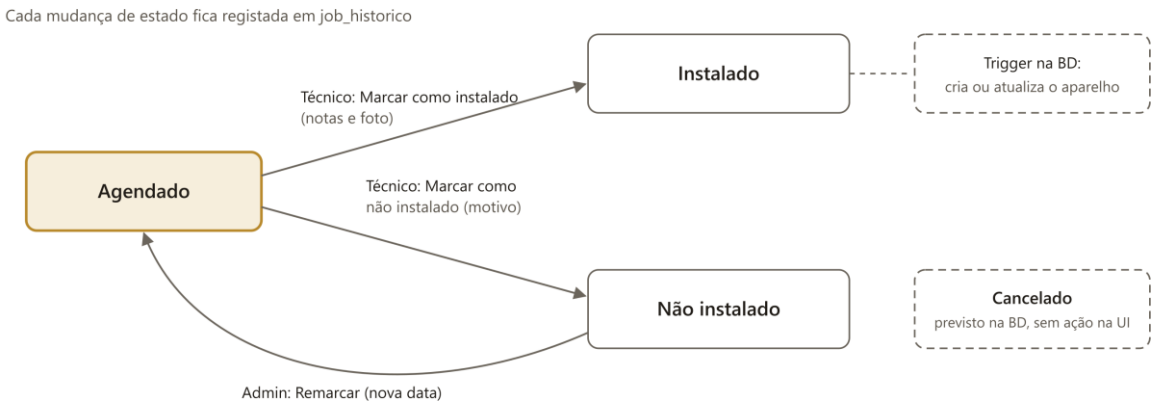


Figura 2 — Estados de um trabalho e quem faz cada transição

O estado **Cancelado** existe na restrição `jobs_estado_check`, mas nenhum ecrã o atribui (ver secção 12).

7. Segurança

As permissões são aplicadas no PostgreSQL e não no browser. Mesmo que alguém altere o frontend ou use a API diretamente com a chave pública, um técnico continua a só conseguir ler e alterar os seus próprios trabalhos.

Perfil	Pode	Não pode
Administrador	Gerir trabalhos, clientes, técnicos, ausências, aparelhos e modelos; ver todo o histórico	—
Técnico	Ver e atualizar os trabalhos a que está atribuído; ver as suas ausências e o histórico dos seus trabalhos; editar o seu perfil	Ver trabalhos de outros técnicos; mudar o seu perfil para administrador; gerir clientes

- **RLS em todas as tabelas.** A função `is_admin()` (`security definer`, `stable`) verifica o perfil de `auth.uid()`. As políticas dos técnicos confirmam a atribuição com `exists (select 1 from job_tecnicos … where tecnico_id = auth.uid())`.
- **Sem escalada de privilégios.** O trigger `profiles_prevent_role_escalation` rejeita qualquer alteração de `role` feita por quem não é administrador.
- **Criação de técnicos sem perder a sessão do admin.** `createSignupClient()` cria um cliente Supabase à parte, sem sessão persistida, usado só para o `signUp`.
- **Storage por pasta.** Cada utilizador só pode escrever o avatar na pasta com o seu `uid`; o envio de fotos de instalação exige sessão iniciada.
- **Segredos fora do repositório.** O `.gitignore` exclui `.env*`. No browser só existem a chave anon e a chave VAPID pública. A chave `service_role` e a chave VAPID privada existem apenas como secrets da Edge Function.

- **Rotas protegidas no cliente.** `ProtectedRoute` redireciona conforme o perfil. Serve só para a experiência de utilização; a segurança real está no RLS.

8. Lógica na base de dados

Há quatro triggers que mantêm dados derivados coerentes, seja qual for o cliente que escreve.

Trigger → função	Quando	O que faz
`on_auth_user_created` → `handle_new_user()`	Após criar um utilizador em `auth.users`	Cria a linha em `profiles` a partir dos metadados (`nome`, `role`, `telefone`)
`jobs_updated_at` → `handle_updated_at()`	Antes de cada `UPDATE` em `jobs`	Atualiza `updated_at`
`jobs_log_historico` → `log_job_historico()`	Após `INSERT` ou `UPDATE` em `jobs`	Regista em `job_historico` o evento `criado`, `estado_alterado` ou `editado`, com o autor e os estados anterior e novo
`jobs_sync_aparelho` → `sync_aparelho_instalacao()`	Após `UPDATE` em `jobs`, quando o estado passa a `instalado`	Instalação de AC, bomba de calor ou solar: cria o aparelho (`on conflict do nothing`, idempotente). Manutenção: atualiza a última e a próxima manutenção do aparelho

As funções de trigger são `security definer` com `search_path = public` fixo. Assim, o histórico e os aparelhos são escritos mesmo quando quem altera o trabalho é um técnico sem permissão de escrita nessas tabelas, e o `search_path` fixo impede que uma função maliciosa seja apanhada no seu lugar.

9. Notificações push e PWA

As notificações avisam cada pessoa quando há algo que lhe diz respeito, mesmo com a app fechada.

Acontecimento	Quem recebe
Trabalho criado ou alterado pelo administrador	Técnicos atribuídos
Técnico marca um trabalho como instalado	Administradores
Técnico marca um trabalho como não instalado	Administradores

1. No Perfil, o utilizador ativa as notificações. O browser cria uma subscrição com a chave VAPID pública (`pushManager.subscribe`), que é guardada em `push_subscriptions`.
2. Quando há um acontecimento, `notifyUsers()` (`src/lib/sendPushNotification.ts`) invoca a Edge Function `send-push` com os destinatários e o texto.
3. A função lê as subscrições com a chave `service_role` e envia-as com a biblioteca `web-push`, assinadas com a chave VAPID privada.
4. O service worker recebe o evento `push` e mostra a notificação. Ao tocar nela (`notificationclick`), abre a app no ecrã indicado.

A PWA usa a estratégia `injectManifest` do `vite-plugin-pwa`: o Workbox faz o pré-cache dos ficheiros gerados e o service worker é escrito à mão. Com `registerType: prompt`, quando há uma versão nova

aparece o aviso "Há uma versão nova da app." com o botão Recarregar, em vez de a app mudar a meio do trabalho.

Ao marcar um trabalho como instalado, o técnico abre a câmara. Alguns telemóveis Android descartam a página em segundo plano nesse momento. Por isso, `pendingAction.ts` guarda o trabalho e as notas em `sessionStorage` e o formulário reabre com o que já estava escrito.

10. Instalação, configuração e deploy

10.1 Requisitos

- Node.js 22 ou superior e npm
- Conta Supabase (o plano gratuito chega) e Supabase CLI, para publicar a Edge Function
- Conta Vercel, para o alojamento

10.2 Base de dados

1. Criar um projeto em supabase.com.
2. No SQL editor, executar `supabase/schema.sql` e depois, por ordem, `migration_v9.sql` a `migration_v13.sql`. Numa base de dados já existente, executar apenas as migrações que ainda não foram aplicadas.
3. Criar o administrador em Authentication → Users → Add user, com os metadados `{"nome": "Nome", "role": "admin"}`. O trigger `handle_new_user` cria o perfil. Os técnicos são criados depois, dentro da app, no ecrã Técnicos.

10.3 Notificações

1. Gerar um par de chaves VAPID: `npx web-push generate-vapid-keys`.
2. Definir os secrets da função: `supabase secrets set VAPID_PUBLIC_KEY=… VAPID_PRIVATE_KEY=… VAPID_SUBJECT=mailto:…`. As variáveis `SUPABASE_URL` e `SUPABASE_SERVICE_ROLE_KEY` já são fornecidas pelo Supabase.
3. Publicar a função: `supabase functions deploy send-push`.

10.4 Aplicação

Copiar `.env.example` para `.env` e preencher as três variáveis:

Variável	Valor
`VITE_SUPABASE_URL`	Project URL (Project Settings → API)
`VITE_SUPABASE_ANON_KEY`	Chave anon `public`
`VITE_VAPID_PUBLIC_KEY`	Chave VAPID pública gerada em 10.3

Comando	Função
`npm install`	Instala as dependências
`npm run dev`	Servidor de desenvolvimento (Vite)
`npm run build`	Verificação de tipos (`tsc -b`) e build de produção em `dist/`
`npm test`	Corre os testes uma vez
`npm run lint`	ESLint

10.5 Publicação na Vercel

1. Importar o repositório na Vercel (framework Vite, comando `npm run build`, pasta `dist`).
2. Definir as três variáveis `VITE_*` em Settings → Environment Variables.
3. Cada push para o ramo principal gera uma nova versão. O `vercel.json` encaminha todas as rotas para `index.html`.

11. Testes automáticos

O projeto tem 56 testes automáticos em 10 ficheiros, escritos com Vitest e Testing Library em ambiente jsdom. Na última execução (16/10/2026) passaram todos. O Supabase é substituído por mocks, por isso os testes não precisam de rede nem de base de dados.

Ficheiro	O que verifica
`lib/manutencao.test.ts`	Conversão mês + dia opcional ↔ data, limites do dia e formatação da próxima manutenção
`lib/notify.test.ts`	Links de WhatsApp (indicativo +351, remoção de espaços e hífenes) e de email (assunto e corpo codificados)
`lib/jobTecnicos.test.ts`	Conversão das linhas de `job_tecnicos` na lista de técnicos de cada trabalho, incluindo perfis apagados
`lib/sendPushNotification.test.ts`	Lista vazia, remoção de destinatários repetidos e erros da Edge Function sem rebentar a app
`lib/initials.test.ts`	Iniciais para os avatares (nomes simples, compostos, maiúsculas, espaços)
`components/CalendarView.test.tsx`	Mês apresentado, cabeçalhos dos dias, semanas completas, navegação, seleção de dia, contagem de trabalhos e destaque de hoje
`components/JobCard.test.tsx`	Cliente e hora, link para o Google Maps, detalhes exclusivos de AC, pré-instalação e foto
`components/JobForm.test.tsx`	Modelos filtrados pelo tipo de equipamento, campos exclusivos de AC e descrição livre para os outros tipos
`components/Chip.test.tsx`	Etiqueta, estilo selecionado e clique
`pages/AdminSettings.test.tsx`	Listagem por tipo, estado vazio, criação e remoção (com confirmação) de modelos

Para correr: `npm test` (uma vez) ou `npm run test:watch` (modo contínuo).

12. Limitações conhecidas e trabalho futuro

- **A Edge Function `send-push` não verifica quem a chama.** Exige sessão iniciada, mas qualquer utilizador autenticado pode pedir notificações para qualquer `userId`. A correção é validar o JWT na função e permitir apenas os destinatários que fazem sentido (admins, ou técnicos do trabalho).
- **Estado Cancelado sem interface.** Existe na base de dados, mas nenhum ecrã o atribui.
- **Migrações manuais.** São executadas à mão no SQL editor. Falta integrá-las no Supabase CLI (`supabase/migrations` + `db push`) e numa pipeline de CI.

- **Testes.** Não há testes end-to-end nem testes às políticas RLS na própria base de dados.
- **Fotos públicas.** Os buckets são de leitura pública: quem tiver o URL de uma foto consegue abri-la. A alternativa é usar URLs assinados.

Funcionalidades previstas: orçamentos com aceitação digital pelo cliente, documentos por cliente, pedidos de assistência e registo de despesas de deslocação dos técnicos.